

RENANCE

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN

COS 101 — Past Questions & Materials

by Renance

Course: COS 101

Institution: Federal University of Petroleum Resources, Effurun

Papers in this pack: 3

Levels: 100

This study pack was compiled and restructured by Renance for COS 101 students at Federal University of Petroleum Resources, Effurun. It brings together 3 past paper(s)/material(s) into one clean, readable document so you can revise without chasing scattered PDFs. Practise these same questions interactively on Renance with instant answers, AI explanations, and full CBT mock exams — over 235,000+ questions across JAMB, Post-UTME, WAEC, NECO and university courses.

Study online: renance.app

Get the app: Google Play — [dev.renance.renance](https://play.google.com/store/apps/details?id=dev.renance.renance)

Generated by Renance AI

Paper 1 of 3 — COS 101 2016/2017

Level 100 • First Semester • PastQuestion

This source paper is a scanned document; its pages are reproduced below.

FEDERAL UNIVERSITY OF PETROLEUM RESOURCES, EFFURUN
COLLEGE OF SCIENCE
DEPARTMENT OF MATHEMATICS/COMPUTER SCIENCE
CSC111 Continuous Assessment Test 2016/2017 SESSION

- This course involves what computers are, how they work and how they are used. Which of these facets does QBasic programmings belong?
- Complete the table below

Flowchart Symbols	Associated QBasic Commands
	INPUT
	PRINT
	IF
	FOR
	GOTO
- _____ error may go unnoticed but _____ error won't because _____
- It makes sense to use a(n) _____ statement before every INPUT statement so that _____
- The expression $2^2 \wedge 3$ in QBasic results in 64 but the programmer expected 256. Rewrite the expression _____
- Complete the table

Mathematical Statement	QBasic Statement
$a = x + y /z$	
$(x^2/\sqrt{y})^3$	

Hint: |x| means absolute value of x; ABS(X) is used in QBasic to find the absolute value of a given number X; while SQR(X) is used to find square root.
- A friend was asked to write a computer program to solve a program, provide an outline of what he needs to do to accomplish the task.
- Using FOR NEXT Loop, write a QBasic program that generates the output below when executed

1	3	4
5	6	7
10	11	12
- Consider the program below and answer the question that follows. The idea is that an investor can invest several times, for each time, he chooses a scheme, either MMM (greedy) or BANK (contented). MMM gives an additional 30% but BANK gives additional 10%. After all investments, the total value of all his investments (which includes all the interest) is displayed.

5 CLS	10 LET T = 0	20 PRINT "Enter amount
30 INPUT D	40 PRINT "Choose Investment scheme (1- MMM, 2- BANK)"	50 INPUT X
60 ON X GOTO 110	70 PRINT "Wrong Entry"	80 GOTO 160
	90 LET T = T + D + (D *)	100 GOTO 120
110 LET T = + D + (D*0.1)	120 PRINT "More Investments? (Y for more, any key for no more)"	130 INPUT Y\$
140 IF Y\$ = "Y" THEN	150 PRINT "GRAND TOTAL = " T	160 END

 - Identify the Five (5) lines that have errors and rewrite them
 - Analyse the corrected program and provide the following statistics

(a) Number of Variables used	(b) Number of Numerical Constants used
(c) Number of Writing operations on memory	(d) Number of reading operations on memory
 - Imagine running the program. When an input is requested use the values: 1000, 1, Y, 2000, 2, and N, one at a time. Draw a box to represent the QBasic screen where QBasic programs are executed. Fill it up so it looks like the actual screen after executing the above program.
 - Draw and label the Flowchart used to generate the program.

www.fupreblog.com

29

www.fupreblog.com

Paper 2 of 3 — COS 101 - Introduction to Computer Science(2014/2015)

Level 100 • First Semester • PastQuestion

Federal University of Petroleum Resources, Effurun – Delta State

First Semester Examination 2014/2015 Academic Session

CSC 111 – Introduction to Computer Science

Time: 2hours 15minutes

1. Which is the most crucial step in programming (A) Understanding the problem (B) Debugging (C) Coding (D) Algorithm/Flowchart
2. When data is transmitted from source to destination so that destination sends back an acknowledgement is called _____ (A) strobe (B) protocol (C) handshaking (D) none
3. What does the “Q” in QBasic stand for (A) Quick (B) Quiet (C) Quit (D) Quite
4. The Wristwatch is a form of Computer. True/False
5. The value of 1100 is _____ in binary (A) 1210 (B) 2010 (C) 1810 (D) 1410
6. The Internet is an example of the LAN network. True/False.
7. The decimal equivalent of 111102 is __ (A) 15 (B) 20 (C) 30 (D) 40
8. The computer is divided into ____ parts (A) 2 (B) 3 (C) 4 (D) 5
9. The combination of two or more arguments to yield either a true or false value is called ____ (A) statement (B) expression (C) proposition (D) none
10. The art of formal reasoning is __ (A) proposition (B) networking (C) logic (D) argument
11. System with only two characters (0 and 1) is __ (A) base 2 (B) Base 8 (C) Base 10 (D) Base 16
12. Symbol is for (a) decision (b) processing (c) sort (d) storage (e) connector (f) merge
13. Symbol is for (a) decision (b) processing (c) sort (d) storage (e) connector (f) merge
14. Symbol is for (a) decision (b) processing (c) sort (d) storage (e) connector (f) merge
15. Symbol is for (a) decision (b) processing (c) sort (d) storage (e) connector (f) merge
16. Symbol is for (a) decision (b) processing (c) sort (d) storage (e) connector (f) merge
17. Symbol is for (a) decision (b) processing (c) sort (d) storage (e) connector (f) merge
18. Subtract 5 and 12 in binary gives us _____ (a)1012 (b) 11012 (c) 1112 (d) 1102
19. Subtract 15 and 5 in binary gives us _____ (a) 10102 (b) 1012 (c) 10112 (d) 11112 (e) none
20. Select the odd one (A) sequence (B) QBasic (C) Selection (D) Loop
21. Represent 0 in the seven segment display (a) 0111111 (b) 1111110 (c) 1101110 (d) 1111111
22. Radio message/ walkie talkie is an example of full duplex transmission mode. True/False

23. Process of entering data into a computer (a) input (b) processing (c) output (d) storage
24. Network of computer in a particular building is ____ (a) sneaknet (b) LAN (c) MAN (d) WAN
25. Microcomputer is one of the two groups of computers. True/False
26. Integrated Circuits is a feature of ____ generation (a) first (b) second (c) third (d) fourth
27. Data transmission one way at a time is ____ mode (a) simplex (b) Half duplex (c) full duplex
28. Data transmission has three stages of connection-transmission-termination True/False
29. Converting analog to digital is (a) modulation (b) demodulation (c) MODEM
30. Conjunction is another name for ____ gate (a) NOR (b) NAND (c) AND (d) OR
31. Computer Literacy is a criteria for selecting microcomputer. True/False
32. Analogue Computers recognize data as physical quantities. True/False
33. Add 10 and 5 in binary gives us ____ (a) 111012 (b) 11112 (c) 10102 (d) 111112
34. A Computer understands algorithms represented through (A) Natural Language (B) Pseudo code (C) Flowchart (D) Programming Language
35. ____ software is used to accomplish specific tasks (a) system (b) application (c) specific
36. ____ sends data from one place to another (a) network (b) Modem (c) data communication
37. ____ replaces manual devices for automatic ones (a) integration (b) automation (c) ergonomics
38. ____ network connects computers across cities (a) sneaknet (b) LAN (c) MAN (d) WAN
39. ____ is the father of computers (a) Babbage (b) Napier (c) Wilkies (d) Pascal
40. ____ is the alignment of text (a) formatting (b) justification (c) delete (d) font
41. ____ is an output device (a) microfilm (b) printer (c) speaker (d) stylus
42. ____ is not an input device (a) keyboard (b) touchscreen (c) printer (d) stylus
43. ____ is most expensive of all sizes of computers (a) Mainframes (b) Micros (c) Supercomputer
44. ____ is an example of computer classified by data type (a) Mainframe (b) Digital (c) MODEM
45. ____ is a characteristics of the computer (a) Program (b) Hardware (c) Accuracy (d) crime
46. ____ introduced logic as used in digital systems (a) Turing (b) Boole (c) Ada (d) Roberts
47. ____ generation of computers date between 1965 - 1971 (a) first (b) second (c) third (d) none
48. ____
49. ____ designed the first microcomputer (a) Intel (b) IBM (c) Roberts (d) Steve Jobs
50. ____ characteristics allows us use computers in all fields (a) crime (b) versatility (c) reliability

Part 2: Answer any One

1a. With Leap year occurring every 4 years. Write a QBASIC program that prints the leap years from 1964 to 2000 (inclusive)

1b. Write a QBASIC program that accepts the full fare and age as well as prints out the fare of an individual if the airline company has a 3 -level price policy:

i. Children under 12 fly free

ii. Individuals from 12yrs to 23yrs fly at student rate (70% of full fare)

iii. Everyone else fly at full rate

2a. Two examination papers are written at the end of the course. The final mark is either the average of the two papers and the class record mark (all weighed equally). Whichever is higher. Write an algorithm and draw the flowchart for the program that accepts the class record mark and the marks of the papers, computes the average and shows either "PASS" if the average is greater than or equal to 50, or "FAIL" otherwise

2b. Write the QASIC program for the algorithm.

Paper 3 of 3 — COS 101

Level 100 • First Semester • Handout

© Allenotor, Oyemade, Enebeli, Kekeje 2016 1

Nested Loops

6/15/2016 Problem Solving with Loops and Nested Loops

Problem Solving with Loops

2

□ We've written loops to go forwards through each character of a string:

```
□ for(int i = 0; i < str.length(); i++)
```

```
System.out.println(str.charAt(i));
```

□ But we can also go backwards!

```
□ for(int i = str.length() - 1; i >= 0; i--)
```

```
System.out.println(str.charAt(i));
```

□ We start i at the str.length() - 1, because the last index of a string is always its length - 1.

□ Then loop if i is greater than or equal to 0 (i >= 0), and decrement i each loop iteration (i--)

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Problem Solving with Loops

3

□ We've seen loops to iterate through each character in a string forwards or backwards.

□ What if we want to find a particular character in the string?

□ We want to have a way to keep track of whether we have found the character we're looking for or not.

□ Is the character found? True or False...

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Finding a Character in a String

4

□ A boolean is perfect for indicating if we've found the character we're looking for or not.

□ The character is either found (true) or not (false)

```
□ boolean found = false;
```

```
for(int i = 0; i < str.length(); i++)
```

```
if( str.charAt(i) == 'x' ){
```

```
found = true;
```

```
}//if
```

□ We can now check to see if the character was in the string:

```
□ if(found)
```

```
System.out.println("Character was found!");
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Searching with Loops

5

```
□ boolean found = false;
```

```
for(int i = 0; i < str.length(); i++)
```

```
if( str.charAt(i) == 'x' ){
```

```
found = true;
```

```
}//if
```

□ There is still a problem here:

□ What if we find the character right away?

□ We'd keep looping until we reached the end of the string!

□ This code would waste a lot of time!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Searching with Loops

6

```
□ boolean found = false;
```

```
for(int i = 0; !found && i < str.length(); i++)
```

```
if( str.charAt(i) == 'x' ){
```

```
found = true;
```

```
}//if
```

□ Now our loop will only run if !found is true:

□ found is false: we haven't found the character yet!

□ As soon as we find the character ('x') we set found = true.

□ !found is then false, so the loop stops as soon as we find 'x'!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Searching with Loops

7

```
//initialize found to false
```

```
boolean found = false;
```

```
//loop until found is true or we reach the end
```

```
//of the string str
```

```

for(int i = 0; !found && i < str.length(); i++)
if( str.charAt(i) == 'x' ){
found = true;
} //if
if(found)
System.out.println("x has been found!");
else System.out.println("x was not found!");

```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Searching with Loops

8

□ Nearly identical code can be used to stop the loop when we find something we don't want:

```

□ boolean accept = true;
for(int i = 0; accept && i < str.length(); i++)
if( str.charAt(i) == 'x' ){
accept = false;
} //if

```

□ Now our loop will only run if accept is true:

□ The string is acceptable so far...

□ As soon as we find something that makes the string unacceptable (the character ('x')) we set accept to false.

□ accept is false, so the loop stops!

□ If the loop reaches the end and accept is still true, then 'x' was never found, so the string str should be accepted!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Classifying Strings with Loops

9

□ We can use the idea of searching for special things inside a string to classify strings.

□ For example:

□ Does the string contain only letters?

□ Does the string contain only numbers?

□ Java has a special helper class to make checking for numbers or letters easier:

□ The Character class.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

The Character Class Methods

10

□ Character.isLetter(char)

- Accepts a single character as a parameter.
- Returns a boolean:
- True if the char is a letter ('a' to 'z' or 'A' to 'Z')
- False if it is not (anything else!)
- Character.isDigit(char)
- Accepts a single character as a parameter.
- Returns a boolean:
- True if the char is a number ('0' to '9')
- False if it is not (anything else!)

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

String Validation

11

- We'll validate a string entered by the user to ensure it contains only letters:

```

boolean accept = true;
for(int i = 0; accept && i < str.length(); i++)
if( !Character.isLetter( str.charAt(i) ) ){
accept = false;
}

```

- This loop will run as long as accept is true.
- The only way for accept to be set to false is if a character that is inside of str is not a letter.
- !Character.isLetter(str.charAt(i)) will only be true if the character at index i is not a letter.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

String Validation

12

- We'll validate a string entered by the user to ensure it contains only digits (numbers):

```

boolean accept = true;
for(int i = 0; accept && i < str.length(); i++)
if( !Character.isDigit( str.charAt(i) ) ){
accept = false;
}

```

- This loop will run as long as accept is true.
- The only way for accept to be set to false is if a character that is inside of str is not a digit.
- !Character.isDigit(str.charAt(i)) will only be true if the character at index i is not a digit.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Problem Solving with Loops

13

□ Let's look at a different problem, using loops to count the number of digits in a number.

□ If the number is a string, this is simple:

□ `String numStr = "482";`

□ `numStr.length()` will give us the number of digits: 3

□ And "-482" would give us 4 instead of 3!

□ What if we don't want to use a String?

□ `int num = 8239;`

□ `num.length()` will not work!

□ Remember, `length()` is a String method. It only works on Strings! `num` is an int, a primitive, so it has no methods!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Counting Digits of an Int

14

□ We'll start off with a counter for the number of digits:

```
int count = 0; //init to 0
```

```
while(num > 0){
```

```
    num /= 10;
```

```
    count++;
```

□ That's all we need! This will take an int and calculate the number of digits it has.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Counting Digits of an Int

15

```
int count = 0; //init to 0
```

```
while(num > 0){
```

```
    num /= 10;
```

```
    count++;
```

□ For example `num = 3925;`

□ First time through the loop: `count = 1, num = 392.`

□ Second time: `count = 2, num = 39.`

□ Third time: `count = 3, num = 3.`

□ Fourth time: `count = 4, num = 0.`

□ The loop stops, because `num > 0` is now false.

□ This also works for negative numbers!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Variable Scope Review

16

□ We've talked about variable scope, and how a variable only exists inside the block of code that it is created in.

□ When the block of code ends, the variable is destroyed.

□ Remember:

□ You can see variables outside of a block of code.

□ You can not see variables inside a block of code.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Variable Scope: Loops

17

□ For example:

```
for(int i = 0; i < str.length(); i++){
if( !Character.isDigit( str.charAt(i) ) ){
accept = false;
} //if
```

```
if(!accept){ //str was not accepted
System.out.println("Error! "
+ str.charAt(i) + " is not a digit!");
```

□ This will cause an error... why?

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Variable Scope: Loops

18

□ For example:

```
for(int i = 0; i < str.length(); i++){
if( !Character.isDigit( str.charAt(i) ) ){
accept = false;
} //if
```

```
} //i is destroyed!
```

```
if(!accept){ //str was not accepted
System.out.println("Error! "
+ str.charAt(i) + " is not a digit!");
```

□ i is declared inside the for loop, so when the loop ends, i is destroyed!

□ The compiler will give an error because i no longer exists when we try to use it inside the if (str.charAt(i)).

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Variable Scope and Loops

19

```

□ for(int i = 0; i < 5; i++){ //new i created
System.out.println();
} //i is destroyed

```

```

for(int i = 0; i < 3; i++){ //new i created
System.out.print("*");
} //i is destroyed

```

□ These for loops will compile, since i is declared inside the first one, it will be destroyed before the second tries to use i.

□ When the second loop attempts to create an int called i, it will be allowed to, since there are no longer any variables by the name of i that exist.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Variable Scope and Nesting

20

```

□ for(int i = 0; i < 5; i++){ //new i created
for(int i = 0; i < 3; i++){ //Error!
System.out.print("*");
System.out.println();
} //i is destroyed

```

□ Now we have the second for loop nested inside the first for loop.

□ i is declared in the first for loop, so it will exist until that loop ends.

□ The second loop tries to declare a new i, but i already exists inside the first for loop, so we get an error.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Variable Scope and Nesting

21

```

□ for(int i = 0; i < 5; i++){ //i is created
for(int j = 0; j < 3; j++){ //j is created
System.out.print("*");
} //j is destroyed
System.out.println();
} //i is destroyed

```

□ We have fixed our original error by simply using a different value for the inner loop's variable.

□ j is also a very common variable to use as a loop index.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Nested Loops

22

□ What does this code do?

```
for(int i = 0; i < 5; i++){ //i is created
for(int j = 0; j < 3; j++){ //j is created
System.out.print("*");
} //j is destroyed
System.out.println();
} //i is destroyed
```

□ Nested loops still behave the same way as regular loops: the loop body is repeated until the condition is false.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Nested Loops

23

□ What does this code do?

```
for(int i = 0; i < 5; i++){ //i is created
for(int j = 0; j < 5; j++){
Loop body
System.out.print("*");
} //j is destroyed
System.out.println();
} //i is destroyed
```

□ No matter what is inside this loop, it will run 5 times.

□ The fact that the loop body has another loop inside isn't important!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Nested Loops

24

□ What does this code do?

```
for(int i = 0; i < 5; Outeri++){
Loop //i is created
for(int j = 0; j < 3; j++){
System.out.print("*");
Loop body
} //j is destroyed
System.out.println();
Outer Loop
```

}//i is destroyed

□ Similarly, the inner loop will loop 3 times when it is run.

□ Each and every time it is run!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Nested Loops

25

```
for(int i = 0; i < 5; i++){ //i is created
```

```
for(int j = 0; j < 3; j++){ //j is created
```

```
System.out.print("*");
```

```
}//j is destroyed
```

```
System.out.println();
```

```
}//i is destroyed
```

□ The inner loop will loop 3 times each and every time it is run!

□ Since the outer loop loops 5 times, that means the inner loop will be run 5 times.

□ The inner loop will loop 3 times each time it is run for a total of $5 * 3 = 15$ times!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Nested For Loops

26

i=0

```
for(int i = 0; i < 5; i++){
```

```
j=0 for(int j = 0; j < 3; j++){
```

```
System.out.print("*");
```

```
}//j is destroyed Output so far:
```

```
System.out.println();
```

```
}//i is destroyed Cursor
```

position

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Nested For Loops

27

i=0

```
for(int i = 0; i < 5; i++){
```

```
j=0 for(int j = 0; j < 3; j++){
```

```
System.out.print("*");
```

```
}//j is destroyed Output so far:
```

```
System.out.println(); *
```

```
}//i is destroyed Cursor
```

```
position
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Nested For Loops

28

```
i=0
```

```
for(int i = 0; i < 5; i++){
```

```
j=1 for(int j = 0; j < 3; j++){
```

```
System.out.print("*");
```

```
}//j is destroyed Output so far:
```

```
System.out.println(); **
```

```
}//i is destroyed Cursor
```

```
position
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Nested For Loops

29

```
i=0
```

```
for(int i = 0; i < 5; i++){
```

```
j=2 for(int j = 0; j < 3; j++){
```

```
System.out.print("*");
```

```
}//j is destroyed Output so far:
```

```
System.out.println(); ***
```

```
}//i is destroyed Cursor
```

```
position
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Nested For Loops

30

```
i=0
```

```
for(int i = 0; i < 5; i++){
```

```
for(int j = 0; j < 3; j++){
```

```
System.out.print("*");
```

```
}//j is destroyed Output so far:
```

```
System.out.println(); ***
```

```
}//i is destroyed
```

```
Cursor
```

```
position
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Nested For Loops

31

```
i=1
for(int i = 0; i < 5; i++){
j=0 for(int j = 0; j < 3; j++){
System.out.print("*");
} //j is destroyed Output so far:
System.out.println(); ***
} //i is destroyed
Cursor
position
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Nested For Loops

32

```
i=1
for(int i = 0; i < 5; i++){
j=1 for(int j = 0; j < 3; j++){
System.out.print("*");
} //j is destroyed Output so far:
System.out.println(); ***
} //i is destroyed
Cursor
position
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Nested For Loops

33

```
i=1
for(int i = 0; i < 5; i++){
j=2 for(int j = 0; j < 3; j++){
System.out.print("*");
} //j is destroyed Output so far:
System.out.println(); ***
} //i is destroyed
Cursor
position
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Nested For Loops

34

```
i=1
for(int i = 0; i < 5; i++){
for(int j = 0; j < 3; j++){
```

```
System.out.print("*");
} //j is destroyed Output so far:
System.out.println(); ***
} //i is destroyed
```

Cursor

position

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Nested For Loops

35

i=2

```
for(int i = 0; i < 5; i++){
j=0 for(int j = 0; j < 3; j++){
System.out.print("*");
} //j is destroyed Output so far:
System.out.println(); ***
} //i is destroyed
```

Cursor

position

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Nested For Loops

36

□ And so on and so on, until finally the outer loop finishes it's last iteration, printing one last line of '*' characters (using the inner loop).

□ The total output of the nested loops:

□ *** //i = 0

*** //i = 1

*** //i = 2

*** //i = 3

*** //i = 4

□ Each line is printed by the inner loop. Once the inner loop finished, a println() moved the cursor to the next line.

□ Once the inner loop started again, it would print the next line and so on...

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Nested Loops

37

□ Another great use for nested loops is to verify user input.

□ We can keep looping until they enter what is desired (the outer loop).

□ We can use a for loop to validate their input string (the inner loop).

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Input Validation with Nested Loops

38

□ We will write a while loop that will continually ask the user for a number. If they enter any other characters that aren't digits, we will not accept their input, and prompt them for a number again.

□ Outer loop structure:

```
□ while(!accepted){
```

```
//prompt for string, check if a number
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Input Validation with Nested Loops

39

□ The inner part of the loop will look familiar:

```
if( input.length() > 0 )//can't be empty!
```

```
accepted = true; //accept for now
```

```
for(int i = 0; accepted && i < input.length(); i++){
```

```
if( !Character.isDigit( input.charAt(i) ) )
```

```
accepted = false;
```

□ Now let's put these two together:

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Input Validation with Nested Loops

40

□ Total validation:

```
boolean accepted = false; //initialize to false
```

```
while(!accepted){
```

```
input = JOptionPane.showInputDialog("Enter a number:");
```

```
if( input.length() > 0 ) //can't be empty!
```

```
accepted = true; //accept for now
```

```
for(int i = 0; accepted && i < input.length(); i++){
```

```
if( !Character.isDigit( input.charAt(i) ) )
```

```
accepted = false;
```

```
}//for
```

```
}//while
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Coming Up...

41

- More problem solving and programming examples.
- More on loops, Strings, and nested loops.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

© Allenotor, Oyemade, Enebeli, Kekeje 2016 1

Input and Output

6/8/2016 JOptionPane, Scanner, and Strings

JOptionPane

2

- JOptionPane are a convenience class built into Java to provide a simple method of displaying messages.
- The compiler needs to know where the JOptionPane is located, so you must include the import line to tell it where to look.
- `import javax.swing.JOptionPane;`
- Import just means that we are adding the JOptionPane to our program from a different location.
- `javax.swing` is the place where Java stores the JOptionPane.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

Importing

3

- Import statements are used so that we can pick and choose which built-in Java classes are added to our programs.
- Some classes are added automatically:
- `System`, `String`, and `Math` for example
- Why not just add all of them all the time?
- When you use a class in your program, it must be added to your program, taking up room!

□ The fewer we have to add, the smaller your program will be.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

Import Statements

4

□ When you import a class into your code, the import statements must be before the public class!

□ It's kind of like declaring a variable, you must declare a variable before you use it.

□ You also need to declare what classes you are importing before you can use them.

□ The import will cause the classes you are importing to be added to your program's Java Byte Code.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

Class Files

5

□ Java code is compiled into Java Byte Code.

□ Java Byte Code is saved into files that are Java Class files.

□ These files have the ".class" extension.

□ You can see them when you compile your code. They will be in the same folder as your ".java" file.

□ Class files are always named after the name of the class that they were created from.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

Class Files

6

□ If your public class' name A1Q1, your java file would have to be A1Q1.java

□ The compiler would create Java Byte Code in a class file called A1Q1.class

□ When you add a class to your program, Java needs to know where that class' .class file is.

□ This is why we import JOptionPane. We are telling java where the JOptionPane.class file is!

□ So there is also a JOptionPane.java file somewhere (in Sun's labs!).

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

JOptionPane Output

7

□ We've already used JOptionPane to display messages in pop-up boxes.

□ `import javax.swing.JOptionPane;`

□ `JOptionPane.showMessageDialog(null, "Hiya!");`

□ The first value passed to `showMessageDialog` method is null. The second is the actual message you wish to be displayed.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

JOptionPane Input

8

□ JOptionPane can also be used to get input from the user!

□ We just need to use a different method other than `showMessageDialog`.

□ `showInputDialog`:

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

JOptionPane: `showInputDialog`

9

□ To call this method, simply use:

□ `JOptionPane.showInputDialog("Message");`

□ Where the value given is a String.

□ It's important to tell the user what kind of input you are expecting!

□ Otherwise they won't know what to type!

This program

could be

expecting

anything!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

JOptionPane: `showInputDialog`

10

□ `showInputDialog` isn't very useful if you don't save the information that the user enters.

□ `showInputDialog` actually returns the whatever the user types in a String.

□ To save the String that the user typed, we'll have to make a variable.

□ But what kind of variable?

□ A String of course!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

```
JOptionPane: showInputDialog
```

```
11
```

□ String input;

□ We can create a String variable similar to how we created ints and doubles.

□ It is important to note that Strings are not primitive data types!

□ String is a built in Java class!

□ More on Strings later, for now, just know that we can create them, and give them values too.

□ String message = "Hiya!"; //message is "Hiya!"

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

```
JOptionPane: showInputDialog
```

```
12
```

```
String input;
```

```
input = JOptionPane.showInputDialog("What is your name?");
```

The program will WAIT here until the user closes the dialog.

□ Once the user closes the dialog, any String they entered will be returned.

□ If we have set up a String variable to capture the input, it will be saved there.

□ Here our input String will contain the user input.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

```
JOptionPane: showInputDialog
```

```
13
```

```
String input;
```

```
JOptionPane.showInputDialog("What is your name?");
```

The program will WAIT here until the user closes the dialog.

□ Here the user will still interact with the dialog in the same way, but the program never saves their input!

□ Now we'll never know their name!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

```
JOptionPane: showInputDialog
```

14

- String input;
- `input = JOptionPane.showInputDialog("What is your name?");`
- Now that we have captured the user input we can use it in our program.
- In this case we could print back whatever they may have entered so they can see it later.
- Printing input is called "echoing input".
- `System.out.println("Hello " + input + " how are you?");`
- If the user entered "Jeff":
- Hello Jeff how are you?

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

JOptionPane and System.exit(0)

15

- You may notice in your programs that use JOptionPane that they seem to take a while to finish.
- This is because JOptionPanes can take a while to properly clean up after themselves.
- If you want your program to finish more quickly, you can use `System.exit(0);`
- This will force Java to end your program! Be careful!
- Only use `System.exit(0);` as the very last line of your main method!
- Anything that comes after that line will be missed!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

Alternatives to JOptionPane

16

- There is another way to capture input from the user.
- You can set up your program to allow the user to type input directly in the console.
- Java provides a convenience class for console input called Scanner.
- Scanner will read in anything the user types, and you can control how you capture it.
- Scanner isn't added by default.
- You need to import it!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

Scanner

17

```
□ import java.util.Scanner;
```

□ This line is required at the beginning of your program if you are going to use Scanner.

□ To use Scanner, you first need to create a Scanner object in your program.

```
□ Scanner keyboard = new Scanner(System.in);
```

□ Here Scanner is the class name, and keyboard is our Scanner variable.

□ Note that keyboard is just a name we made up, it has no significance, we could have called it bob.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

Scanner

18

```
□ Scanner keyboard = new Scanner(System.in);
```

□ To create a Scanner object, we need to introduce a keyword: new

□ new is a special Java keyword that indicates we want an object created.

□ Java creates the object in memory and returns a pointer to its location in memory.

□ Don't worry about specifics of memory for now. That's more important in later courses.

□ Remember that the new keyword creates objects for you.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

Scanner

19

```
□ Scanner keyboard = new Scanner(System.in);
```

□ We need to tell Java what the Scanner is going to scan for us!

```
□ Here we use new Scanner(System.in);
```

□ This is the Scanner constructor.

□ This just means it creates the object.

□ The Scanner constructor requires the value of what we want scanned for input.

□ We want to use System.in

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

Scanner

20

- `Scanner keyboard = new Scanner(System.in);`
- `System.in` is the console itself. The above line is creating a new `Scanner` object that will scan the console (`System.in`) for input.
- Now that we have a `Scanner` object (our keyboard variable), we can start capturing input that the user types.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

Scanner Input Methods

21

- `Scanner` has many different input methods:
- `nextLine()` //grab a line of text as a `String`
- `nextInt()` //grab text as an `int`
- `nextDouble()` //grab text as a `double`
- `nextLong()` //grab text as a `long`
- There are more `nextDataType` methods that parse the text entered as a particular `dataType`.
- For the most part we'll use `nextLine()`

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

Scanner `nextLine()` Method

22

- `Scanner keyboard = new Scanner(System.in);`
- `String name;`
- `System.out.println("Enter your name: ");`
- `name = keyboard.nextLine();`
- `System.out.print("Hello " + name + " how are you?");`
- Here the `print` is split up on two different lines, this is ok as long as you don't break up the quotations (`Strings`) or variable names.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

Scanner Console Input

23

Here the user is prompted for input using `System.out.print`

The user has typed text ("`David`"). The `Scanner` has not captured the text yet,

because it is waiting for the “Enter” key to finish the line.

The user has pressed the “Enter” key. The string they typed (“David”) is saved and printed back to the console.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

Dealing with Input

24

□ Now that we can capture Strings, it would be nice to be able to do things with them besides printing them.

□ If we want to get numbers from Strings, we’ll have to learn how to extract them.

□ Remember that like primitive data types:

□ You can’t just save one type as another.

□ It’s the same with Strings and other data types!

□ We need a way to convert Strings to numbers!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

String Parsing

25

□ `int distance; //distance in kilometres`

□ `String numStr = “5”;`

□ `distance = numStr; //Error!`

□ Distance is an int, but numStr is a String. It doesn’t matter that numStr looks like a number!

□ It is still a String!

□ What we need is a way to read through a String and create a number from it!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

String Parsing

26

□ String is a built-in Java convenience class.

□ Java has other convenience classes to make parsing of numbers easier (and other things).

□ Integer for ints, Double for doubles, etc.

□ There is a catch, we need to be sure what we’re parsing out.

□ If we parse something that isn’t in the String, our

program will crash!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

String Parsing

27

□ If we parse something that isn't in the String, our program will crash!

□ This requires some trust in what the user will enter.

If we expected an int, but they entered a double, the program will crash.

□ In later courses you'll learn how to prevent your programs from crashing from bad input.

□ For this course we'll prompt the user for what we want, and hope they don't mess it up!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

String Parsing

28

□ String input = "10";

□ To extract an integer from a String, we use a Integer method called parseInt.

□ parseInt takes a single value as its parameter, which is a String.

□ This should make sense, we need a String to parse!

□ parseInt returns the int that was parsed

□ Or crashes if the String wasn't an int!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

String Parsing: parseInt

29

□ String input = "10";

□ int num;

□ num = Integer.parseInt(input);

□ Note: You must use "Integer." before the parseInt method, just like you need to use "Math." before Math's pow method.

□ In the above code, our int variable num is assigned the result of the Integer.parseInt method, in this case, the integer 10.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

String Parsing: parseDouble

30

```
□ String input = "2.99";  
□ double num;  
□ num = Double.parseDouble(input);  
□ Note: You must use "Double." before the  
  parseDouble method, just like you need to use  
  "Math." before Math's pow method.  
□ In the above code, our double variable num is  
  assigned the result of the Double.parseDouble  
  method, in this case, the double 2.99  
© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016
```

Integer and Double

31

```
□ Wait, what are Integer and Double?  
□ Weren't we using int and double instead before?  
□ Integer and Double are both Java classes. They  
  provide extra methods that allow us to work more  
  easily with ints and doubles.  
□ Like parseInt and parseDouble for example!  
□ It's important to note that int and double are  
  primitive data types, not classes!  
© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016
```

String Parsing

32

```
□ There is also ways to parse floats, longs, etc.  
□ Float.parseFloat(someString)  
□ Long.parseLong(someString)  
□ These methods all use the same format. The class  
  name is first, then a period, the method name, then  
  the String to be parsed .  
□ parseFloat returns a float, parseLong returns a long  
  etc...
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

Parsing an Int Example

33

```
□ String input;  
□ int age = 0;  
□ input = JOptionPane.showInputDialog("How old are  
  you?");
```

```
int age = Integer.parseInt(input);
```

Here input captures the user input from a JOptionPane.

age is then parsed from the input string using Integer.parseInt

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

String Details

34

We've used Strings a little bit, but what are they exactly?

The String class is a class built into Java. You do not need to import it. It is used so often that it is included automatically.

The name "String" comes from the fact that you are "stringing" a number of characters together.

So a String is just a group of characters!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

String Details

35

The String "Hi! abc" is actually represented in memory as a String of characters, as shown below. Each box represents a character in the String "Hi! abc"

H i ! a b c

All characters are actually represented by numbers!

72 105 33 32 97 98 99

These are the ASCII numeric values of the characters above. Java uses Unicode sometimes as well.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

The Char Data Type

36

To create single characters in Java, we can use the char data type (short for character).

Each char can only hold a single character.

To make a char literal use single quotations.

```
char c = 'a';
```

```
c = '1';
```

```
c = 'ab'; //Error! Too many characters!
```

- There are some special characters called escape sequences which have special behaviours.
- They look like two characters, because a \ is needed to indicate that they're special.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

Special Characters (Escape Sequences)

37

- \n
- Newline character: Creates a new newline...
- \t
- Tab character: Creates a tab...
- \\
- The backslash character, since \ tells Java that this is a special character, to actually print a \, you need to use \\
- \'
- Single quote
- \"
- Double Quote

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

Special Characters

38

- Note: These are all printing single characters!
- `System.out.print('\n');`
- Moves cursor to next line
- `System.out.print('\t');`
- Tabs the cursor
- `System.out.print('\\');`
- Outputs \
- `System.out.print('');`
- Outputs '
- `System.out.print('');`
- Outputs "

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

Special Characters and Strings

39

- We know that Strings are just made up of a bunch of characters.
- Using special characters we can make our Strings more useful!
- String heading = "Name\tAddress";

- What would this look like if we printed it?
- Name Address
- String daveAddr = "Dave\tCS-123";
- Dave CS2-123
- Notice that the tabs line up! We can make our output look more professional.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

Special Characters and Strings

40

- If we print our Strings one after another:
- Name Address
- Dave CS2-123
- Sue 123 Fake Street
- Now it looks like we have a rudimentary table.
- We can also use the '\n' character to make printing spaces easier.
- System.out.println("1\n2\n3");
- Prints 1, then a new line, 2, then a newline, and finally 3, and then the println itself will add the last newline.
- Output: 1
- 2
- 3

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

String Methods

41

- Since Strings are a class, they also have some methods to make them more useful.
- charAt(index)
- Returns the character in the String at the given index.
- length()
- Returns the number of characters in the String.
- toLowerCase()
- Converts the String to lowercase letters.
- toUpperCase()
- Converts the String to uppercase letters.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

Using Methods

42

- When we want to use a class' methods, we

generally need to have an instance of that class.

- An instance is something we have created. An instance of a class is known as an object. So a String object must exist before we use its methods.

- Unlike with Math methods, where we never make a Math object. We just call Math.pow(3,2), Math.sqrt(9) etc.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

Using Methods

43

- String.toLowerCase() for example doesn't make sense. What String do we want converted to lowercase?

- We need to have an actual object, in this case, a String!

- String str; //we created a String object

- Now to use a String method with our str variable, we type our variable name followed by a period.

- When we use a method on an object, we say we are calling method on that object.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

Using Methods: String Example

44

objectName.methodName(parameters);

The name of the object The name of the Any parameters that that we want to call our method that we want to are required for the method on call method call

```
String str = "I AM BEING QUIET";
```

```
//str exists, we can call methods
```

```
str.toLowerCase(); //no parameters
```

```
//str is now "i am being quiet"
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

String's length Method

45

- Remember, we need to have an actual String to use these methods.

- What does that mean?

- You can't just call it with String.length()

- What are you asking the length of? You first need to create a

String to use these methods on!

- String word = "antidisestablishmentarianism";
- System.out.println("Word length: " + word.length());
- This works because word exists, so it makes sense that we can ask how long it is.
- Word length: 28

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

String's charAt Method

46

- charAt also requires an existing String to use.
- String numbers = "012345";
- System.out.print(numbers.charAt(3));
- Prints 3
- Side Note: In programming, we start our indices at zero! This makes code easier to use. We'll get into this much more later, but for now just remember, the first index of a list is always 0!
- String msg = "Hello!";
- System.out.print(msg.charAt(0));
- Prints H

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

Coming Up

47

- We will talk a great deal more about Strings in the next few lectures.
- More exciting however is conditional programming!
- Conditional programming means we can choose what pieces of our program are run.
- This makes them more flexible, and much more powerful!
- We'll start in with if statements next lecture!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/8/2016

© Allenotor, Oyemade, Enebeli, Kekeje 2016 1

If-Statements

Conditionals and String methods

Announcements

2

- A1 is due this week (...) at midnight!
- You must submit online using the piazza

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Code as a Path

3

□ Up to this point, we've only

int amount = "1000"; talked about straight forward programs.

□ Our programs have executed from start to

accountBalance += amount; finish, executing each line of code.

□ Here our code can only

add and subtract amount

accountBalance -= amount;

from accountBalance.

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Fork in the Road

4

□ What we need is a way to choose which lines of code are run, and which are not.

□ This is called conditional programming.

□ Java and many, many other languages use if statements.

Is this a deposit?

If yes If no

accountBalance += amount; accountBalance -= amount;

© Allenotor, Oyemade, Enebeli, Kekeje 2016

If Statements

5

□ Java uses the keyword if for programming if statements.

□ If statements can evaluate a condition (or conditions) and determine if they are true or false.

□ The line directly under the if statement will be executed if and only if the condition is true.

□ Syntax:

□ if(condition)

//Code to run if condition is true

© Allenotor, Oyemade, Enebeli, Kekeje 2016

If Statement Example

6

□ Here is a simple example. The 99.99 value in the if statement represents the purchase price.

```
//The user has $0.12 in their account.
```

```
int accountBal = "0.12";
```

```
if( accountBal > 99.99 )
```

```
System.out.println("Purchase Approved!");
```

```
if( accountBal < 99.99 )
```

```
System.out.println("Not enough money!");
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016

If Statement Example

7

□ In the previous example, we had two if statements.

□ One checked to see if there was more than enough money in the bank account, and printed a message if there was.

□ The other checked to see if there wasn't enough money, then printed a message if that was true.

□ There is a simple way to combine this into a single structure, by introducing the else keyword.

© Allenotor, Oyemade, Enebeli, Kekeje 2016

If-Then-Else

8

□ Java and many other languages use something called an if-then-else structure to control code execution more easily.

□ If the if statement is true, then execute some code, else (otherwise) execute some different code.

□ The else keyword is used to create this if-then-else structure, like in the following example.

© Allenotor, Oyemade, Enebeli, Kekeje 2016

If-Then-Else

9

```
//The user has $0.12 in their account.
```

```
if( accountBal > 99.99 )
```

```
System.out.println("Purchase Approved!");
```

```
else System.out.println("Not enough!");
```

□ In this example the condition in the if statement is evaluated (accountBal > 99.99).

□ If this is true, then “Purchase Approved” will be printed. If this condition is false, then “Not enough!” will be printed instead.

© Allenotor, Oyemade, Enebeli, Kekeje 2016

If-Then-Else

10

```
//The user has $0.12 in their account.
```

```
if( accountBal > 99.99 )
```

```
System.out.println(“Purchase Approved!”);
```

```
else System.out.println(“Not enough!”);
```

□ It is very important to remember that only one of the above print statements will happen.

□ The condition inside the brackets will determine which of the two will be run.

□ This is the whole reason we use if statements (and if-else statements)! It’s to control what pieces of our program run, and what don’t!

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Comparison Operators

11

□ In the previous examples we used the simple comparison of `accountBal > 99.99`

□ This uses the greater than operator: `>`

□ There are a number of comparison operators in Java that can be used to compare numbers.

□ Sometimes they can be used with non-numeric data as well, but for this course we’ll just be using them for numeric comparisons.

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Comparison Operators

12

□ `>`

□ Greater than

□ `<`

□ Less than

□ `>=`

□ Greater than or equal to

□ `<=`

□ Less than or equal to

□ ==

□ Equal to. Note: Equals is NOT '=', a single '=' is the assignment operator

□ !=

□ Not equal to. This is the opposite of == (equals)

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Comparison Examples

13

□ 2 < 5 //true

□ 2 <= 5 //true

□ 2 > 5 //false

□ 2 == 2 //true

□ 2 <= 2 //true

□ 2 >= 2 //true

□ 2 != 2 //false

□ 2 == (1+1) //true

□ 3 == (Math.sqrt(9)) //true!

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Comparisons

14

□ As seen in the last example, comparisons can be fairly complex. As long as each side evaluates to a number, it is acceptable.

□ (8 - 7) == (9 % 4) //true, 1 == 1

□ (8 + 2) > "robot" //Error! Can't compare
//Strings and ints!

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Result of a Comparison

15

□ A comparison results in a value of either true or false.

□ There is no in-between! It is either one or the other!

□ This is actually very heavily based on 0's and 1's...

□ Java even has special keywords true and false.

□ true and false are literals.

□ Java actually has a special data type that can only hold true or false values.

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Boolean Data Type

16

□ The boolean data type is a primitive data type that can only hold the values true or false.

□ boolean found;

□ found = true;

□ found = false;

□ We can use booleans directly inside our if statements as the conditions!

□ if(found)

System.out.println("Found it!");

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Booleans

17

□ The keywords true and false can also be used directly... but this is almost never done!

□ It doesn't make sense... if(true) will always be executed no matter what! It's always true!

□ It is very common to use booleans directly in an if-else statement:

□ if(deposit)

balance += amount;

else balance -= amount;

□ We can also combine multiple conditionals inside a single if statement

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Combining Conditions

18

□ There are 3 special operators in Java called boolean operators.

□ These operators can be used to combine different truth values.

□ They are from general logic:

□ AND

□ OR

□ NOT

□ In fact, computers only use these 3 operations to run! (But they use billions of them!)

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Logical (Boolean) AND: &&

19

- In Java, the AND operator is && (ampersand)
- When && is used to combine two booleans, the result is true if and only if both are true:
- false && false //false
- false && true //false
- true && false //false
- true && true //true

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Logical (Boolean) OR: ||

20

- In Java, the OR operator is ||
- Two “pipes” from using Shift and the ‘\’ key
- When || is used to combine two booleans, the result is true if at least one is true:
- false || false //false
- false || true //true
- true || false //true
- true || true //true

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Logical (Boolean) NOT: !

21

- In Java, the NOT operator is !
- The ! operator will negate any boolean it is used on. If it was true, it will become false. False will become true:
- !false //true
- !true //false

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Boolean Expressions

22

- Don’t be confused with the boolean data type.
- It simply holds a truth value. It can get that truth value directly from a literal:
- approved = true;
- Or from the result of a boolean expression:
- approved = (accountBal > purchasePrice);
- In the above example, the boolean expression is evaluated, and if accountBal is greater than purchasePrice, the boolean approved will be

assigned true otherwise, false.

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Combining Booleans

23

□ We can make more elaborate and more useful if statements by combining boolean expressions.

□ If we want to write a credit checking program to see if we should issue credit cards, we might want to check to make sure the user is 18, and that they have at least \$1000.

```
□ if( age >= 18 && accountBal > 1000 )
```

```
System.out.println("Credit card approved!");
```

```
else System.out.println("Rejected!");
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Combining Booleans

24

□ Maybe we want to check to see if a number is between a range of values (e.g. 0 and 10).

```
□ if( num >= 0 && num <= 10 )
```

```
System.out.println("In range!");
```

```
else System.out.println("Out of range!");
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Combining Booleans

25

□ We might want to check to make sure a number is not zero:

```
□ if( num > 0 || num < 0 )
```

```
System.out.println("Not Zero!");
```

□ There is another easier way to check to see if something is not equal to zero:

```
□ if( num != 0 )
```

```
System.out.println("Not Zero!");
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Combining Booleans

26

□ Here we are using NOT (!) to negate the boolean hasAccount and also checking to see if accountBal is less than or equal to 0.

```
□ if( !hasAccount || accountBal < 0 )
```

```
System.out.println("Rejected!");
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016

If-Else-If

27

□ We can also have else statements that check their own conditions.

□ These are done using an else if structure, where the else doesn't happen unless the if statement is evaluated as true.

□ This is a way to chain if statements together so we can have elaborate ways to decide which code to run.

© Allenotor, Oyemade, Enebeli, Kekeje 2016

If-Else-If Example

28

```
if( termMark < 50 )  
    grade = "F";  
else if( termMark < 60 )  
    grade = "D";  
else if( termMark < 70 )  
    grade = "C";  
else if( termMark < 80 )  
    grade = "B";  
else if( termMark < 90 )  
    grade = "A";  
else grade = "A+";
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016

If-Else-If Wrong Example

29

```
if( termMark < 90 ) □ This doesn't work!  
    grade = "A"; □ You need to be careful what  
else if( termMark < 80 ) order you put your if  
    statements in!  
    grade = "B";  
□ In this case, anyone with a  
else if( termMark < 70 ) grade less than 90 will get  
    grade = "C"; an A!  
else if( termMark < 60 ) □ Even if they had a term mark  
    grade = "D"; of 1, they'd still get an A!
```

else if(termMark < 50) □ This is because the first if statement checks for < 90, if grade = "F"; that's true, it will not check else grade = "A+"; any of the others!

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Blocks of Code

30

□ We've seen blocks of code already, though you may not have realized it.
□ Anything withing curly braces { } is a block of code.
□ A block of code is treated as a single entity, it is all run like a miniature program.

```
public static void main(String[] args){  
//everything between these curly braces is  
in //the same block of code. This is a special  
//block of code, since it's in the main  
method.
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Blocks of Code

31

□ You can define a block of code wherever you want, but to keep with programming standards, we will only use them for classes, methods, and if statements.
□ What do you think is printed?

```
if( 3 > 0 ){  
System.out.print("Hello!");  
System.out.print("Goodbye!");  
else System.out.println("Robots Are Awesome!");
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Blocks of Code

32

□ What do you think is printed?

```
if( 3 > 0 ){  
System.out.print("Hello!");  
System.out.print("Goodbye!");  
else System.out.println("Robots Are Awesome!");
```


□ This would print: Hello!Goodbye!
□ The if statement is true, so everything inside the if

statement is run. This means that both print statements are printed!

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Blocks of Code Styles

33

□ There are two common styles for defining blocks of code (I use the first one generally):

□ `if(condition){`

`//Do stuff if condition is true`

□ `if(condition)`

`//Do stuff if condition is true`

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Blocks of Code Styles

34

□ It doesn't matter which style you use, choose whichever you like best.

□ Just be sure to stick with one style!

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Comparing Strings

35

□ We would obviously like to compare Strings, to check to see if a name is what we're looking for etc.

□ `if( name == "Dave" )`

`System.out.println("You are awesome!");`

□ This does not work!

□ And no it's not just because I am not awesome!

□ You can't compare Strings with comparison operators.

□ You need to use a special String method

© Allenotor, Oyemade, Enebeli, Kekeje 2016

String's compareTo method

36

□ String has a built-in method called `compareTo` that will compare two strings and return an integer.

□ `compareTo` returns:

□ 0 if the two strings are identical

□ A value < 0 if the first string is alphabetically less than the other

□ A value > 0 if the first string is alphabetically greater than the other

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Using compareTo

37

□ if(name.compareTo("Dave") == 0)

System.out.println("Hi dave!");

□ if(name.compareTo("Dave") < 0)

System.out.println("Less than Dave");

□ if(name.compareTo("Dave") > 0)

System.out.println("Greater than Dave");

© Allenotor, Oyemade, Enebeli, Kekeje 2016

Coming up...

38

□ More on if statements and booleans

□ More String methods, more on compareTo

□ More blocks of code

□ Nested code blocks

□ Nested if statements

© Allenotor, Oyemade, Enebeli, Kekeje 2016

© Allenotor, Oyemade, Enebeli, Kekeje 2016 1

Nesting

6/15/2016 Nested Ifs and Code Blocks

String Input Checking

2

□ We learnt last class about a String method called compareTo.

□ compareTo returns:

□ 0 if the two strings are identical

□ A value < 0 if the first string is alphabetically less than the other

□ A value > 0 if the first string is alphabetically greater than the other

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

String Input Checking

3

□ If the user doesn't enter any data,

JOptionPane.showInputDialog will return the empty string ("").

□ If we compare the input to the empty string and they are equal, then we know the user didn't enter anything.

□ If they are not equal, it means that the user input is not the empty string, and therefore they have typed some information for us to use in our program.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

String Input Checking

4

□ We can check to see if the user entered data or not using compareTo.

□ We will compare the user's input string to the empty string.

□ "" is the empty string, it's still a String, but there is nothing inside it!

□ If the user doesn't enter any data,

JOptionPane.showInputDialog will return the empty string ("").

□ So will the Scanner method nextLine()

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

String Input Checking

5

```
String input = "";
```

```
input = JOptionPane.showInputDialog("Enter your name: ");
```

```
//check to see if input is empty string
```

```
If(input.compareTo("") == 0){
```

```
System.out.println("User did not enter any data");
```

```
else {
```

```
System.out.println("Process the input string...");
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

String Input Checking

6

□ We can also check the size of the String input, if it has a length of zero, it is empty.

```
String input = "";
```

```
input = JOptionPane.showInputDialog("Enter your name: ");
```

```
//check to see if input is empty string
if( input.length() == 0 ){
System.out.println("User did not enter any data");
else {
System.out.println("Process the input string...");
}
}
© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016
```

String Input Checking

7

- We can now determine if the user has entered any data.
- We can place all our code that processes the string inside the if statement so that it only happens if the user actually entered data.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Alternatives to compareTo

8

- compareTo is great for checking to see if two strings are the same (if the result of compareTo is equal to 0).
- compareTo is also very useful to determine which strings are alphabetically greater or smaller than each other.
- It is a little awkward, comparing to 0 etc, especially when all we want to do is check to see if two strings are equal.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

equals

9

- There is also a String method called equals which simply returns a boolean value.
- True if they are equal, false if not.
- With equals, case matters!
- "abc" is not the same as "ABC"
- "aBc" != aBC" etc...
- equals will only check if two strings are the same, it can not compare them to see which one is greater than the other.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

equalsIgnoreCase

10

□ There is also a String method called equalsIgnoreCase which returns a boolean value.

□ True if they are equal, false if not.

□ With equalsIgnoreCase, case doesn't matter!

□ "abc" is the same as "ABC"

□ "aBc" == aBC" etc...

□ equalsIgnoreCase will only check if two strings are the same, it can not compare them to see which one is greater than the other.

□ There is also a compareToIgnoreCase

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

String Equivalencies

11

□ String a = "hello";

□ String b = "world";

□ a.equals(b); //false

□ a.equals("hello"); //true

□ a.equalsIgnoreCase("HELLO"); //true

□ b.equalsIgnoreCase(a); //false

□ b.equals("world"); //true

□ b.equalsIgnoreCase("WoRLd"); //true

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Checking User Input

12

□ Many times we want to ask the user to make a decision, often a simple "yes" or "no".

□ We will read in their response as a String, then compare it to "yes" and "no" to determine what the user desires.

□ We can also check to see if they only entered a single letter, so we'll check for "y" and "n" as well.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Checking User Input

13

□ //Ask the user if they want to play a game

□ //assume that input is read in from the user

if(input.equalsIgnoreCase("yes")

|| input.equalsIgnoreCase("y")){

```
//play the game
else System.out.println("Goodbye!");
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Checking User Input

14

□ It would be nice to also check to see if the user entered anything. We can do this by combining the logic into a single if statement.

□ We want to run the program if:

□ The user entered something AND

□ User input is "yes" OR user input is "y"

□ This is trickier than it might sound. We need to use brackets to make sure that the OR is treated as a single expression.

□ This is just like using brackets in arithmetic.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Boolean Precedence with Brackets

15

```
If(input.length() > 0
&& (input.equalsIgnoreCase("yes")
|| input.equalsIgnoreCase("y"))){
//play the game
else System.out.println("Goodbye!");
```

□ The if statement has is an AND (&&) of one boolean expression: `input.length() > 0` and another:

□ `(input.equalsIgnoreCase("yes") || input.equalsIgnoreCase("y"))`

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Boolean Precedence with Brackets

16

□ `(input.equalsIgnoreCase("yes") || input.equalsIgnoreCase("y"))`

□ This is evaluated as a whole expression, similarly to how an arithmetic expression is evaluated with the brackets first.

□ $(2+3) * 5 = 25$, since $(2+3)$ is done first.

□ So for the boolean expression, if `input = "y"`:

□ `( false || true )` would be what is evaluated, giving

a final result of:

□ `true!`

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Boolean Precedence with Brackets

17

□ So for the total if expression, if input = "y":

```
If(input.length() > 0
```

```
&& (input.equalsIgnoreCase("yes")
```

```
|| input.equalsIgnoreCase("y"))
```

□ This results in:

```
If(true && (false || true))
```

□ Which is further evaluated to be:

```
If(true && (true))
```

□ Finally resulting in true! They want to play the game!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Nesting

18

□ We've seen a few examples of if statements so far with some code that is run if the if statement is true.

□ The code we run can be an if statement itself!

□ This allows multiple levels of checking, allowing us to make more sophisticated programs.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Nested Ifs

19

```
if( input.length() > 0
```

```
&& ( input.equalsIgnoreCase("yes")
```

```
|| input.equalsIgnoreCase("y") ) ){
```

```
//play the game
```

```
if( winner )
```

```
System.out.println("YOU WIN!!");
```

```
else System.out.println("Loooooooooooooser!");
```

□ Here the bold if statement (the inner if) is inside the first if (the outer if).

□ It will only be run if the first (outer) if is true.

□ It won't even be checked otherwise!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Nested Ifs

20

□ These are called nested ifs, □ If A, B, and C are all true, because the blocks of code "CBA" will be printed.

are nested inside each other. □ If A and B are true, "BA"

```

if( A ){ will be printed.
if( B ){ if A is true, only "A" will
if( C ){ be printed.
System.out.print("C");
If A is false, it doesn't
System.out.print("B");
matter what the rest are,
nothing will be printed!
The other if statements are
System.out.print("A");
inside of A's!
}

```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Nested Code Blocks

21

```

We'll do much more with nested code blocks later, once
we get into loops.
Remember that an else can have a block of code
associated with it as well:
And of course, nested blocks of code!
if(A)
System.out.print("A");
else {
num++;
System.out.print(num);
}

```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Coming up

22

```

Next week: Loops!
We can start making our code repeat lines as many
times as we want.
In our game example, we can ask the user if they
want to try again.
We can use loops to count, go through items in a list,
etc.
They are the last major feature of a programming
language.

```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

© Allenotor, Oyemade, Enebeli, Kekeje 2016 1

Loops

6/15/2016 While and For loops

Increment and Decrement Operators

2

□ We have already seen combined operators such as

`+=` and `-=`:

□ `num += 1;` //same as `num = num + 1;`

□ `num -= 1;` //same as `num = num - 1;`

□ There is an even easier way to increment and decrement if you only want to add or subtract 1.

□ `num++;` //same as `num = num + 1;`

□ `num--;` //same as `num = num - 1;`

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Increment and Decrement Operators

3

□ These operators are used all the time, because quite often we only want to add or subtract 1 from a number.

□ An index for example, if we want to go through items in a list one at a time (index 0, 1, 2, 3, etc).

□ Aside: C is an old language that C++ was built on.

The name “C++” is actually a (lame) programming joke, because C++ is meant to be the next step for C...

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Proper Indentation

4

□ Indentation is an important part of our programming standards.

□ Programming standards are not meant to just make your code look good, they are to help you avoid errors!

□ The easier it is to read, the easier you'll find the bugs!

□ Indentation (tabs) are used to clear up if statements, blocks of code, and especially nested blocks (and ifs!)

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Indentation Styles

5

```

if-while-else-etc{ if-while-else-etc
stuff-inside {
stuff-inside stuff-inside
} stuff-inside
else {
else
stuff-inside
stuff-inside

```

Similar indentation examples can be found on the course webpage under the “Java Standards” link, as well as many other programming standards.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Indentation and Nesting

6

□ Indentation works the same way when you have nested ifs, loops, or code blocks.

□ You still indent the lines inside by one tab.

□ Each new level of nesting is indented one more tab

```

if(num > 0){ //depth 0
//depth 1
num++;
if(num < 10){ //depth 1 still
//depth 2
System.out.println("Done!");
} //end this if at depth it started at (1)
} //end this if at depth 0

```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Looping

7

START □ We’re used to what we call sequential programs.

int num = 0; □ Sequential means that each line follows the one before it, in
num++; sequence.

□ This is straightforward, but not
print(num); entirely useful.

END

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Looping

8

START □ With a loop, we can
check a condition, and if
int num = 0; it is true, we go back to
the start of the loop.

num++;

□ Once the condition is

True

false, we end the loop

print(num);

and continue in our

Condition

program.

False

END

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Looping Conditions

9

□ A loop will continue to loop until the condition is
false.

□ We need to make sure the condition will eventually
become false.

□ Otherwise our loop would run forever, and our program
would never end!

□ When you are creating loops, the looping condition
is as important as the code inside the loop!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Loop Body

10

□ The loop body is the code that is executed repeatedly.

□ The loop body can be a single statement:

□ while(condition)

System.out.print("Hi"); //loop body

□ The loop body is more commonly a block of statements:

□ while(condition){

//statement 1

//statement 2

} //end of the loop body

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

The While Loop

11

□ A while loop is a structure that will check a boolean condition (just like we use in if statements) and loop while that condition is true.

□ while is a keyword in Java used to make these while loop structures.

```
while(num > 0){ //this will loop until num is  
num--; //less than 0. If num is  
} //positive, how many times?
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

While Loops

12

□ While loops continue looping until their looping condition is false.

□ The condition is a boolean expression, so it can be very elaborate, as we've seen with if statements.

```
while( num > 0 && num % 2 == 0 ){  
System.out.println(num);  
num--;
```

□ What will be printed?

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

While Loops

13

□ We need to watch for the most common loop error: infinite loops!

```
□ while(true)  
System.out.println("I'll never stop!!");
```

□ It is easy to mess up and accidentally make a loop that will never end.

```
□ while(num <= 10)  
System.out.println(num);  
num++;
```

□ This last one loops infinitely too! Why?

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

While Loop Indentation

14

```
□ while(num <= 10)  
System.out.println(num);  
num++;
```

□ This while loop doesn't contain the `num++`; line of code! It just looks like it does because it is not properly indented!

□ `while(num <= 10)`

`System.out.println(num);`

`num++;`

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

While Loop Code Blocks

15

□ Make sure that you use proper curly braces to designate your loops' code blocks!

□ Indentation will help make it clear what code belongs to which block too!

□ Now we'll write the previous example with proper indentation and braces so it won't be an infinite loop:

□ `while(num <= 10){`

`System.out.println(num);`

`num++;`

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

While Loops and Input Validation

16

□ While loops are great structures for validating input.

□ Keep reading input from the user until they enter the type of input you are looking for.

□ For example, keep looping until the user enters 'q' or quit:

`while(!input.equals("q")){`

`//keep doing stuff`

`input = keyboard.nextLine();`

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Priming a Loop

17

□ Often we want to loop based on user input.

□ We will need to prime the loop.

□ Priming means doing a single step just before the loop, where that step is done inside the loop as well.

□ If we want to loop until the user enters 'q', but we

don't want to perform anything in the loop, we'll have to first read in from the user, then start the loop.

□ We'll still need to read in from the user inside the loop!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Priming a Loop

18

```
//prime the loop by reading in input once to check
```

```
//before the loop starts
```

```
System.out.println("Enter name (q to quit):");
```

```
name = keyboard.nextLine();
```

```
while( !name.equals("q") ){
```

```
System.out.println(name);
```

```
System.out.println("Enter name (q to quit):");
```

```
name = keyboard.nextLine();
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Do-While Loop

19

□ In the previous example, we'll only print their name if they didn't enter 'q'.

□ Sometimes we want the loop to run at least once no matter what.

□ There is a special kind of while called a do-while.

□ The do-while loop executes the body of the loop first, then checks after to see if it should continue looping.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

The do-while Loop

20

□ Note: you must end a do-while with a semi-colon.

□ Do-whiles aren't used a great deal, but they are quite useful when you want to perform the loop body at least once before checking the looping condition.

```
do {
```

```
System.out.println("Enter name (q to quit):");
```

```
name = keyboard.nextLine();
```

```
System.out.println(name);
```

```
} while( !name.equals("q") );
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

The For Loop

21

□ There is one other type of loop called the for loop.

□ for is a keyword in Java just like while used to create a for loop structure.

□ A for loop has a very different structure than a while loop.

□ It still checks for a condition, but it also has 2 other pieces:

□ An initialization step,

□ And an update step.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

The For Loop

22

□ The for loop syntax:

```
for(initialization; condition; update)
```

```
//statement or code block
```

□ Initialization is where you will often declare a variable to use within the for loop itself.

□ An extremely common use is an index to use to go through a list (or go through each character in a string using charAt)

```
for( int index = 0; condition; update)
```

```
//statement or code block
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

For Loop Initialization

23

□ When using an index, most programmers use 'i' as the index:

```
for(int i = 0; condition; update)
```

```
//statement or code block
```

□ Here the int i is initialized to 0 in the for loop's initialization step.

□ Initialization is done only once!

□ Initialization only happens one time, at the very beginning of the for loop.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

For Loop Condition

24

□ The for loop's condition is just a boolean expression like an if statement or a while loop's condition:

```
□ for(int i = 0; i < 5; update)
//statement or code block
```

□ Here the condition is $i < 5$.

□ That means the loop will execute its statements until $i < 5$ is false.

□ Won't this go on forever?

□ Yes, unless we update the i variable!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

For Loop Update

25

□ The for loop's update is a way to ensure that the loop will eventually end:

```
□ for(int i = 0; i < 5; i++)
//statement or code block
```

□ Here the update step is $i++$.

□ This means that after each loop iteration, i will be incremented by 1.

□ Now the for loop's condition ($i < 5$) will eventually become false, and the loop will stop.

□ It will run 5 times, when $i = 0, 1, 2, 3, 4$.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

For Loops

26

□ None of the pieces of a for loop are required.

□ Though you will almost always want them all!

□ If you don't have a condition, the loop will run forever!

```
□ for( ; ; )
```

```
System.out.println("I'll never stop!");
```

□ This for loop goes on forever, since there is no way to stop it!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

For Loops

27

□ You can have more than one initialization in a for loop, though this is rarely, if ever used.

- It just looks sloppy, and is hard to read:
- `for(int i = 0, j = 0; condition; update)`
- Your condition can be a boolean expression as elaborate as you like, and this is quite useful.
- `for(int i = 0; !found && i < input.length(); i++)`
`if( input.charAt(i) == '!' )`
`found = true;`
- Here the loop goes through a String called input.
- The loop will stop if found is set to true, or when i reaches the end of the input string.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

For Loops

28

- The update step can also have multiple things updated, but it also is rarely ever done, because it also makes the loop harder to read:
- `for(int i = 0, j = 0; i <= 10 && j < 5; i += 2, j++)`
`System.out.println(i + j);`
- It is important to note that the update step can have any expression in it, not just a ++ or --:
- `for(int i = 0; i < 10; i = i * 5 - i)`
- Be careful! This would loop infinitely!
- `0 * 5 - 0` is always 0! i would always be 0!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Aside: Declaring Multiple Variables On One Line

29

- Just a quick note, as you have seen in previous slides with for loops, you can declare multiple variables of the same type on one line:
- `int i = 0, j = 0; //i and j are ints initialized to 0.`
- `String input, name; //input and name are Strings`
- `double x, y, z; //x, y, and z are all doubles`
- `int i, j = 4, k; //i, j, and k are all ints, j is init to 4`

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Scope and For Loops

30

- `for(int i = 0; i < 5; i++){`
`System.out.print(i + " ");`
`}//end of for`

```
System.out.println("i: " + i);
```

□ What is printed in the for loop?

□ 0 1 2 3 4 //the loop ends with $i = 5$, because
//5 < 5 is false!

□ What is printed in the line after the for loop?

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Scope and For Loops

31

```
for (int i = 0; i < 5; i++) {
```

```
System.out.print(i + " ");
```

```
} // end of for
```

```
System.out.println("i: " + i);
```

□ What is printed in the line after the for loop?

□ Nothing! The program won't compile!

□ It will give a compile time error:

□ cannot find symbol symbol : variable

i

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Scope and For Loops

32

```
for (int i = 0; i < 5; i++) {
```

```
System.out.print(i + " ");
```

```
} // end of for
```

```
System.out.println("i: " + i);
```

□ cannot find symbol symbol : variable i

□ How can it not find i?! It's right there, isn't it?

□ Actually i was declared inside the for loop.

□ I can't be seen outside of the for loop!

□ Once the for loop ends, i is destroyed

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Scope

33

□ Scope is an advanced concept which we'll go into in detail next class.

□ The basic idea is that a variable only exists in the scope where it is declared.

□ This means that if a variable num is declared in a block of code, once that block of code ends, num

ceases to exist!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Scope

34

□ Example showing the scope of the doubles outsidelf and insidelf using an if statement:

```
public static void main(String[] args){
    double outsidelf = 4; //declared outside the if
    if( true ){ //this if will always have it's code run
        outsidelf = 10;
        int insidelf = 42;
    } //end of if

    System.out.println(outsidelf); //Ok, prints 10
    System.out.println(insidelf); //Will not compile!
    //insidelf is not declared in this block of code!
} //end of main
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Coming Up...

35

- More on while and for loops
- Using String methods with while and for loops
- Much more on variables and their scope.
- Nested Loops!
- Loops that are inside of other loops!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

© Allenotor, Oyemade, Enebeli, Kekeje 2016 1

Variable Scope; Strings

6/15/2016 Scope, Strings and Loops, String Methods

Types of Errors

2

□ Now that we are getting into some more advanced programming concepts, it is important that we can recognize different types of errors in our programs so we can fix them.

□ In Java (and many other languages) there are three main error types:

□ Compile time errors,

- Run time errors,
- and Logical errors

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Compile Time Errors

3

- Compile time errors are simply errors that happen when you try to get your program to compile.
- They are syntax errors, because the compiler won't accept your code.
- Its syntax is incorrect, so it won't compile.
- Compile time / syntax errors are generally easiest to fix.
- Read the compiler error message, and try to make it happy.
- Once the compiler errors are fixed, the program has proper syntax and can be run.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Run Time Errors

4

- Run time errors occur when the program is already running.
- Run time errors can crash your program!
- These can be very difficult to find!
- For example, you have a loop that becomes an infinite loop if certain conditions happen (possibly based on user input).
- You will need to run your programs and play around with them to try to find all the run time errors and fix them.
- Print statements help a great deal!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Logical Errors

5

- A logical error will not stop your program from compiling, and it will not crash your program (generally).
- Logic errors are just errors in how the program operates.
- It is producing the wrong results!
- Sometimes for simple reasons, like doing the wrong

calculation. What do you mean you should get paid more than \$0.10 an hour?

□ You'll need to test different inputs, print out variable values and run your program many times to make sure that it is working as intended.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Semantic Errors

6

□ Logical errors are semantic errors as opposed to syntactic errors.

□ The grammar is correct, but what you are saying isn't what you intended to say.

□ "My cat is a tuba. The sky is crunchy."

□ Both are grammatically correct sentences...

□ In programming semantics is how you put your code together (with correct syntax) to solve a problem.

□ Just like in the above English sentence example, proper syntax can still lead to semantic errors because the code doesn't do what you intended.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Variable Scope Details

7

□ Now that we know the main error types, let's talk more about variable scope.

□ Not understanding variable scope can lead to all three of these error types, so it's important that you understand how it works.

□ We'll review it more once we start talking about making our own methods, but you should know how it works now too.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Variable Scope

8

□ We often declare our variables at the beginning of our program (the start of the main method).

□ When a variable is declared, it comes into existence, so the rest of the program can use it.

□ However, a variable is only visible in the same scope as it was created in.

□ A variable declared inside the main method is visible everywhere in the main method.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Variable Scope

9

```
public static void main(String[] args){
//main's scope blue, if's scope red
int x = 0;
if(x >= 0){ //start of if's scope
x *= 3; //x is still visible
} //end of if's scope
x++; //still main's scope
} //end of main's scope
```

□ Here x is declared inside of main's scope, so it is visible everywhere in main, including inside the if statement.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Variable Scope

10

```
if(x >= 0){ //start of if's scope
x *= 3; //x is still visible
} //end of if's scope
```

□ Here we see that the x is visible, even though we've started a new internal scope with the if statement.

□ The x is blue to indicate that it comes from the main method.

□ There are different levels of scope with nested code blocks (in this example, main is the outer code block, if is the inner code block)

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Levels of Scope

11

□ When you have a block of code nested inside another, the outer block of code can NOT see variables declared inside the inner block of code.

□ This means that any variables created inside the inner block are destroyed when that code block ends.

□ The outer block will not be able to reference those variables, because they don't exist anymore!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Levels of Scope

12

- From inside an inner block of code (an if statement for example) all the outer block's variables are still visible.
- Like in our previous example, the if statement can still access the variable x, because x is declared in an outer block of code.
- Inner Scope has full access to all the variables declared in it's containing or outer scope.
- What the hell does all this mean?!
- The solution, as it so often is in Computer Science, is to draw boxes! Many, many boxes!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Scope Analogy: Boxes

13

```
public static void main(String[] args){
//main's scope
int x = 0;
while( x <= 10 ){ //while's scope
int num = 9;
if( x == num ){ //if's scope
String msg = "x == num";
System.out.println(msg);
} //end of if's scope
} //end of while's scope
} //end of main's scope
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Scope Analogy: Boxes

14

```
public static void main(String[] args){
//main's scope x is declared inside main's
int x = 0; scope, so main, the while and
the if can see it.
while( x <= 10 ){ //while's scope
int num = 9;
if( x == num ){ //if's scope
String msg = "x == num";
System.out.println(msg);
} //end of if's scope
```

```
//end of while's scope
```

```
//end of main's scope
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Scope Analogy: Boxes

15

```
public static void main(String[] args){
```

```
//main's scope
```

```
int x = 0;
```

```
while( x <= 10 ){ //while's scope
```

num is declared inside the while's

```
int num = 9; scope, so the while and if can see
```

it. Main cannot.

```
if( x == num ){ //if's scope
```

```
String msg = "x == num";
```

```
System.out.println(msg);
```

```
//end of if's scope
```

```
//end of while's scope
```

```
//end of main's scope
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Scope Analogy: Boxes

16

```
public static void main(String[] args){
```

```
//main's scope
```

```
int x = 0;
```

```
while( x <= 10 ){ //while's scope
```

```
int num = 9;
```

```
msg is declared inside if( x == num ){ //if's scope
```

the if's scope, so the if

```
String msg = "x == num";
```

can see it. Main and

```
the while cannot. System.out.println(msg);
```

```
//end of if's scope
```

```
//end of while's scope
```

```
//end of main's scope
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Scope Access: Outer to Inner

17

□ Using boxes as our analogy to talk about scope, if you are inside a box (a scope) and you have

another box inside that box (a deeper level of scope, like the while inside main) you can't see into the box.

□ General rule is:

□ You can't see inside of the box.

□ You can see outside of the box (from inside it).

□ More on that soon!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Outer to Inner Scope Access

18

```
public static void main(String[]
```

```
From inside the mainargs){
```

```
method, we can't see
```

```
//main's scope anything inside of the while loop. So if any
```

```
int x = 0; variables are declared inside, they are
```

```
invisible to us out in main!
```

```
}//end of main's scope
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Outer to Inner Scope Access

19

```
public static void main(String[] args){
```

```
//main's scope
```

```
int x = 0; If we are inside of the while's scope,
```

```
we can't see inside of the if
```

```
while( x <= 10 ){ //while's scope
```

```
statement, so any variables
```

```
int num = 9; declared inside the if are invisible to
```

```
us! They are also invisible to main,
```

```
since the while loop is invisible to
```

```
??? main!
```

```
}//end of while's scope
```

```
}//end of main's scope
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Viewing Outer Scope From Inner

20

□ If we are inside of an inner scope (the if statement)

we can see variables declared inside the outer scope(s) (the while, main).

□ This is because we can see outside of the box.

□ So we can see all of the while's variables, and main's, but they can't see ours.

□ Remember, you can't see inside a box, only outside!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Inner to Outer Scope Access

21

```
public static void main(String[] args){
//main's scope
int x = 0;
while( x <= 10 ){ //while's scope
int num = 9;
if( x == num ){ //if's scope
String
The if can see outside of itsmsg
scope =
to "x == num";
the whileSystem.out.println(msg);
loop's scope.
} //end of if's scope
} //end of while's scope
} //end of main's scope
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Inner to Outer Scope Access

22

```
public static void main(String[] args){
//main's scope
int x = 0;
while(
The while loop can x
see <=
out of 10 ){ to //while's scope
its scope
main's
intscope.num = 9;
if( x == num ){ //if's scope
String msg = "x == num";
System.out.println(msg);
} //end of if's scope
} //end of while's scope
} //end of main's scope
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Inner to Outer Scope Access

23

```
public static void main(String[] args){
//main's scope
int x = 0;
while( x <= 10 ){ //while's scope
int num = 9;
if( x == num ){ //if's scope
The if can see the while's String
scope, and the
msgwhile
= "x == num";
can see main's scope, so the if can see main's
System.out.println(msg);
scope too!
} //end of if's scope
} //end of while's scope
} //end of main's scope
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Scope and Nesting

24

- Remember:
- The inner scope can see outside to the outer scope's variables.
- The outer scope can't see inside its inner scopes.
- General rule:
- You can see out.
- You can not see in!
- It's almost as if each scope was a box made of one-way mirrors, with the mirrored side on the outside of each box. If you are inside, you can see through the mirror and out. If you are outside, you can't see inside the mirrored box.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Strings

25

- We've talked a bit about Strings already.
- String is a Java class that can hold many different characters (a "string" of characters).

□ Strings also have many methods that allow us to perform different operations on their data (the characters inside them).

□ Let's quickly review two methods we'll use a lot (and you'll need on for Assignment 2):

□ `length()`

□ and `charAt(int)` //where int is an index

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

String's `length()` Method

26

□ Remember, we need to have an actual String to use these methods.

□ You can't just call `String.length()`

□ It doesn't make sense. What are you asking the length of? You first need a String with data to use these methods on!

□ `String word = "abcdef";`

□ `System.out.println("Word length: " + word.length() );`

□ This works because word now has data (characters), so it makes sense that we can ask how long it is.

□ Word length: 6

□ Note: The empty string gives `length()` of 0.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

`length()` and Loops

27

□ A String's length is needed if we want a loop that will run once for each character in the string.

□ For example, if the string was length 4, we'd want the loop to run 4 times.

□ With a for loop, we can start our index (i) at 0, and increment it by 1 each time.

□ As long as our condition stops the loop once i is equal to the length of the String, we'll stop at the end.

□ `for(int i = 0; i < str.length(); i++)`

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

`length()` and Loops

28

□ `for(int i = 0; i < str.length(); i++)`

□ Hey, wait a minute, shouldn't it be:

□ `i <= str.length()` ? //less than or equal?

- No! The first index in a string is always 0!
 - We want i to start at the first character's index: 0.
 - We want i to end at the last index, so the loop body will run only when `i < str.length()`
 - `str.length()` is the number of characters, not the index of the last character!
 - The index of the last character is `str.length() - 1`
- © Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

String's `charAt(int)` Method

29

- `charAt` also requires an existing String to use.
- `String letters = "abcde";`
- `System.out.print(letters.charAt(3));`
- Prints 'd'
- Remember: In programming, we start our indices at zero!
- `String msg = "Hello!";`
- //indices: 012345
- `System.out.print(msg.charAt(0));`
- Prints 'H'

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

String's `charAt(int)` Method

30

- We need to make sure that our loop index will always be in the range of valid characters.
- That is, it won't be negative, or have a value that is greater than or equal to `str.length()`
- This is especially important when using `str.charAt(i)`.
- If i is not a valid index of str, then our program will crash!
- Example from a TextPad error message:

Exception in thread

"main"java.lang.StringIndexOutOfBoundsException:

String index out of range: 33

at java.lang.String.charAt(String.java:687)

at Stuff.main(Stuff.java:15)

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Strings and Loops

31

- We'll start i at the first index of the first character (0).
- We'll print the character at index i during each loop

iteration.

□ i will be incremented after each loop iteration.

```
for(int i = 0; i < str.length(); i++)
```

```
System.out.println("charAt(" + i + "): "
```

```
+ str.charAt(i) );
```

Printed so far:

```
str = H e l l o !
```

```
charAt(0): H
```

```
Indices: 0 1 2 3 4 5
```

```
i=0
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Strings and Loops

32

□ We start i at the first index (0).

□ We'll print the character at index i during each loop iteration.

□ i will be incremented after each loop iteration.

```
for(int i = 0; i < str.length(); i++)
```

```
System.out.println("charAt(" + i + "): "
```

```
+ str.charAt(i) );
```

Printed so far:

```
str = H e l l o !
```

```
charAt(0): H
```

```
charAt(1): e
```

```
Indices: 0 1 2 3 4 5
```

```
i=1
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Strings and Loops

33

□ We start i at the first index (0).

□ We'll print the character at index i during each loop iteration.

□ i will be incremented after each loop iteration.

```
for(int i = 0; i < str.length(); i++)
```

```
System.out.println("charAt(" + i + "): "
```

```
+ str.charAt(i) );
```

Printed so far:

```
str = H e l l o !
```

```
charAt(0): H
```

```
charAt(1): e
```

Indices: 0 1 2 3 4 5 charAt(2): l

i=2

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Strings and Loops

34

□ We start i at the first index (0).

□ We'll print the character at index i during each loop iteration.

□ i will be incremented after each loop iteration.

```
for(int i = 0; i < str.length(); i++)
```

```
System.out.println("charAt(" + i + "): "
+ str.charAt(i) );
```

Printed so far:

```
str = H e l l o !
```

```
charAt(0): H
```

```
charAt(1): e
```

Indices: 0 1 2 3 4 5 charAt(2): l

```
charAt(3): l
```

i=3

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Strings and Loops

35

□ We start i at the first index (0).

□ We'll print the character at index i during each loop iteration.

□ i will be incremented after each loop iteration.

```
for(int i = 0; i < str.length(); i++)
```

```
System.out.println("charAt(" + i + "): "
+ str.charAt(i) );
```

Printed so far:

```
str = H e l l o !
```

```
charAt(0): H
```

```
charAt(1): e
```

Indices: 0 1 2 3 4 5 charAt(2): l

```
charAt(3): l
```

```
charAt(4): o
```

i=4

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Strings and Loops

36

- We start i at the first index (0).
- We'll print the character at index i during each loop iteration.

- i will be incremented after each loop iteration.

```
for(int i = 0; i < str.length(); i++)
```

```
System.out.println("charAt(" + i + "): "
+ str.charAt(i) );
```

Printed so far:

```
str = H e l l o !
```

```
charAt(0): H
```

```
charAt(1): e
```

```
Indices: 0 1 2 3 4 5 charAt(2): l
```

```
charAt(3): l
```

```
charAt(4): o
```

```
i=5 charAt(5): !
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Strings and Loops

37

- We start i at the first index (0).
- We'll print the character at index i during each loop iteration.

- i will be incremented after each loop iteration.

```
str.length() is 6. i is now 6, so
```

```
i < str.length() is now false,
```

```
for(int i = 0; i < str.length(); i++)
```

```
System.out.println("charAt(" + i and the loop
+ "): " stops!
```

```
+ str.charAt(i) );
```

i has gone off the end

```
str = H e l l o !
```

of the String str!

There are no more

Indices: 0 1 2 3 4 5 characters to access!

```
i=6
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

String Concatenation

38

- Concatenation is a fancy name for "sticking a bunch of things together".
- For example, a train is made from multiple train cars

concatenated together...

□ Definition from Answers.com:

□ "To connect or link in a series or chain."

□ When we talk about String concatenation, we are talking about sticking two strings together to make a new string out of both.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

String Concatenation

39

□ For example if we have two Strings:

String start = "Hello ";

String end = "World!";

String combined;

combined = start + end;

//combined is now "Hello World!"

'+' is the String

concatenation

operator.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

String Concatenation

40

□ We've already seen this with our print statements:

□ System.out.println("Hello " + "World" + '!');

□ In the above statement we are actually creating a single string by concatenating two separate strings and a single character.

□ You can also use the concatenation operator (+) to add single characters to a string.

□ The final string is: "Hello World!"

□ Since println takes a single string as its parameter, it needs to concatenate them into one string first, and then print it.

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Concatenation with +=

41

□ += is a short hand way of adding a string or character to an existing string.

□ str += "some string"; //is the same as:

□ str = str + "some string";

□ For example:

```
String str = ""; //start as an empty string.
```

```
str += 'a'; //str is now "a"
```

```
str += 'b'; //str is now "ab";
```

```
str += "solutely!"; //str is now "absolutely!"
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

`+=` and the Empty String

42

□ When we are adding to a string, we often want to start it off being empty.

□ This way we can create a new string character by character!

□ Didn't we just talk about loops and strings, and how to go through and get each individual character from a string?

□ Now we can build a new string from those characters!

□ We can change their order, or just take the odd ones, etc...
It's really up to us!

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Loops, Strings, and Concatenation

43

```
String msg = "tphqics{ [izs& 5aA xs9e3cHrYett%!"
```

```
String decoded = ""; //start empty so decoded will
```

```
//only contain the characters
```

```
//that we add to it
```

```
//This loop will only add the characters from the
```

```
//String msg to decoded if they have even indices.
```

```
for(int i = 0; i < msg.length(); i++)
```

```
if( i % 2 == 0 ) //if i is even
```

```
decoded += msg.charAt(i); //add character
```

```
System.out.println("decoded message: " + decoded);
```

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Result:

44

□ msg was: "tphqics{ [izs& 5aA xs9e3cHrYett%!"

□ decoded is: "this is a secret!"

© Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016

Coming up...

45

- Nested Loops!
 - For real this time!
 - More on Strings
 - Problem solving and many programming examples!
- © Allenotor, Oyemade, Enebeli, Kekeje 2016 6/15/2016